

Multitask Active-Learning

Shubham Chitnis^{1,*} and Paula Branco²

¹Student, Department of Chemical Engineering, Indian Institute of Technology Bombay

²Assistant Professor, School of Electrical Engineering and Computer Science, University of Ottawa

*190020033@iitb.ac.in

ABSTRACT

Example Abstract. Abstract must not include subheadings or citations. Example Abstract. Abstract must not include subheadings or citations. Example Abstract. Abstract must not include subheadings or citations. Example Abstract. Abstract must not include subheadings or citations. Example Abstract. Abstract must not include subheadings or citations. Example Abstract. Abstract must not include subheadings or citations. Example Abstract. Abstract must not include subheadings or citations.

Introduction

The Introduction section, of referenced text expands on the background of the work (some overlap with the Abstract is acceptable). The introduction should not include subheadings.

Previous Work

Multitask Learning

A typical method to approach problems involving multiple targets is to deal with them as standalone tasks. This method fails to leverage any inter-domain information already present in the data. Multitask Learning as a learning paradigm was formalized in 1997 [1] to make predictions to multiple related tasks simultaneously and, in the process, also improve generalization. Since then, the field has seen immense applications of MTL in various data modalities, including text [2,3], images [4,5], and audio [6,7]. A typical example of MTL would be the vision system of a self-driving cars. Frames from multiple cameras placed onboard are used to perform object detection, image segmentation, depth estimation, etc. The survey on MTL undertaken by Yu Zhang and Qiang Yang [8] goes into the details of different sub-approaches within MTL and a detailed classification of works they came across in the literature.

In our work, we deal with a special case of MTL wherein we have the same data corresponding to all the different tasks. This does overlap with the paradigm of multi-label or multi-output learning, which doesn't involve the case in MTL, where separate tasks could have different kinds of data. Another vital thing to note is that multi-label learning uses a single cost function to optimize the model performance across all labels jointly. In MTL, one can use different cost functions targeting various tasks.

Class Imbalance

A recurring theme that is seen in most machine learning datasets is class imbalance within the data. Quite often, some classes are more commonly seen as compared to others. When models are trained on such data directly without applying any prior heuristics, we get a final model that favors predicting the majority class. From the model's perspective, it is doing well since this leads to a low loss function value; however, it has no practical use. A lot of work has been done proposing clever sampling strategies [9–11] or better loss functions [12] to tackle imbalance.

Most of the work on data imbalance has been directed towards a single target (binary or multiclass). When considering a multi-target scenario, many conventional approaches fail since we want to simultaneously balance individual targets while avoiding disturbing the balance for others. There are particular works addressing the imbalance for multiple targets. Still, they deal with the imbalance within labels at times, also called global imbalance (i.e., some labels occur more often than others), ignoring the local class imbalance [13,14]. . For example, consider a scenario where we have to simultaneously diagnose the existence of two respiratory diseases (let us call them Disease A and Disease B) using the patient's chest x-ray image. Now Disease A is more commonly seen than Disease B, so frequent diagnostic tests are run for Disease A and more data is available for the same. This is what we call Global Imbalance. It is also fair to assume that there would be more data available for patients without a disease as compared to those with it. This kind of imbalance within different classes of the same target is local imbalance. We consider datasets with equal availability of labels for all targets removing global imbalance altogether.

Active Learning

As stated in the work by Burr Settles [15], active Learning is a “Family of machine learning methods which may query the data instances to be labeled for training by an oracle.” These methods often tend to match or outperform the conventional approaches using fewer labeled data instances. Many recent works are emerging in the field since the active learning setup that involves a lot more unlabeled data, and relatively fewer labeled data points is seen in most applications [16–18].

A study to evaluate the use of active learning to tackle the problem of imbalanced datasets was done in [19]. The authors proposed an efficient system of selecting useful instances from a small data pool. In [20], Torres et al. also dealt with using active learning on an imbalanced dataset by creating a random forest-based model to determine the probabilities of unlabelled network traffic data instances and choosing the best ones to label for intrusion detection systems.

Multi-Task active learning is a recent domain where research is ongoing to search for better solutions. In [21], Zhang et al. discuss a strategy for multi-task active learning where the uncertainty rankings of unlabeled data are averaged to select the most informative instances for different tasks. A similar approach of averaging the uncertainty rankings was adopted in [22] for linguistic annotations. Zhang et al. in [23] proposed a framework utilizing the inconsistencies of predictions between different tasks and treating them as constraints. In [24], a multi-task active learning framework is proposed that uses the support vector preservation criterion for uncertainty evaluation and sampling.

The only resembling work that we came across which attempted to tackle the issue mentioned above of local class imbalance in a multi-label setting was from Liu et al. [25]. The authors came up with two novel sampling approaches, MLSOL and MLUL, and discussed the performance boost obtained on an ensemble of classifiers. However, the authors concluded by documenting results for various multi-label datasets and did not integrate their approaches into a framework like active learning.

Methods

Dataset Description

In this work we use 10 multi-target datasets comprising of targets that are binary, multiclass or a combination of both. Different imbalance ratios are considered from balanced and slightly imbalanced to extreme data imbalance. In Table 1 we report the datasets that we have worked with alongside their targets, number of attributes and instances.

We measure the degree of imbalance within different targets of a dataset using the metric *Imbalance Ratio* (IR). There are two ways in which we define the imbalance ratio. IR_2 is the ratio of the number of instances of the majority class to that of the minority class. While IR_2 is more commonly seen in the literature, we should be note that when defining it this way, we neglect every class other than the majority and minority classes. To take the effect of other classes into consideration as well, we define IR_1 as the average of the ratio of number of instances of the majority class to that of every other class excluding itself.

In other words, consider a target having k classes $\{C_1, C_2, C_3, \dots, C_k\}$ and let $n_j \in N, j = 1, 2, 3, \dots, k$ denote their corresponding number of instances. Then we can define imbalance ratios as follows:

$$IR_1 = \frac{\text{Maximum}(N)}{\text{Minimum}(N)} \quad \text{and} \quad IR_2 = \text{Maximum}(N) \cdot \text{Average}\left(\frac{1}{N_i}\right), \quad \text{where} \quad N_i = N - \text{Maximum}(N)$$

Before using these datasets, several preprocessing steps were performed. Some of the steps like removing duplicate data points, dropping unknown or null values and getting rid of unique identifiers such as customer id were common to every dataset. Other than these, in the *Smoking* dataset, we got rid of the attribute *Oral* a single value was repeated throughout the column. Also for using *NewBuilt* as a target of the *Paris Housing* dataset, we had to suppress the attribute *Made* to avoid any sort of data leakage.

A few changes were also made to the targets themselves. The target *Imdb score* initially had a range from 0 through 10. It was then grouped into three final classes (Good: $8 \leq \text{score} < 10$; Average: $5 \leq \text{score} < 8$; Bad: $0 \leq \text{score} < 5$). Another target *Content Rating* of the IMDB dataset, had a few classes relabelled to avoid high similarity between them. Since we are strictly dealing with the classification task, if numerical variables are to be treated as targets, we would need to convert them into categorical first. The numerical target *Gross* was thus posed as a binary variable using 15 million as the rough estimate of the average gross income of films and denoting whether the movie in question exceeded the average (target value 1) or didn't (target value 0). Finally, we fixed some typos within the data instances and removed the *Other* class from the *Customer Type* target in *Bank Intention* dataset; making it a binary variable with just the *New* and *Returning* customers left.

Baseline Implementation

We start off with a baseline implementation of the Multitask Active Learning framework. The setup involves splitting the dataset into three sets: initial training, pool, and test. We reserve 100 samples for the initial training and 1000 for the test. Both of these stay the same across all the datasets. The budget is also kept fixed since we assume that irrespective of the dataset that

Dataset	Columns	Instances	Label 1		Label 2		Label 3	
			Name (Type)	IR_1 IR_2	Name (Type)	IR_1 IR_2	Name (Type)	IR_1 IR_2
IMDB	45	3737	Imdb score (M)	13.196 15.395	Content Rating (M)	40.701 105.938	Gross (B)	1.908
Mushroom	23	5644	Class (B)	1.618	Population (M)	11.767 41.538	Habitat (M)	11.994 38.938
Census Income	15	45194	Income (B)	3.033	Marital Status (M)	122.011 657.75	Work Class (M)	277.553 1584.81
Bank Marketing	21	3090	Target (B)	7.351	Personal Loan (B)	5.095	Housing Loan (B)	1.204
Shopper Intention	18	12124	Revenue (B)	5.408	Visitor Type (B)	6.161	Special Day (M)	48.22 70.617
Anuran Calls	25	7195	Family (M)	25.066 65.0	Genus (M)	23.499 61.029	Species (M)	16.813 51.147
Telco Churn	20	7021	Churn (B)	2.781	Contract (M)	2.444 2.616	Payment (M)	1.519 1.55
Paris Housing	17	10000	Category (B)	6.905	New Built (B)	1.004	Storage Room (B)	1.012
Smoking	25	44552	Smoking (B)	1.725	Tartar Status (B)	1.251	Dental Caries (B)	3.683
Flight Feedback	23	129487	Satisfaction (B)	1.302	Customer Type (B)	4.46	Class (M)	3.838 6.609

Table 1. Table describing the datasets used and their corresponding targets

we are working with, we only get to make a fixed number of queries to the oracle. Here, by budget, we mean the maximum number of samples that we can send to the oracle (or an annotation team, in an industry setting) for labeling. A budget would be decided based on factors like time constraints, computational requirements, or the annotation team size. During each query step, we train our classifiers on the training data and use them to search for 45 samples from the pool that they are least confident about (15 corresponding to each target). The oracle then provides us with the true labels for these samples and they are added to our training data for future steps. We repeat this for 20 steps and plot the trends of accuracy and f1 score for all 3 targets. For this implementation, we use a Random Forest classifier which is an ensembling method comprising multiple Decision Trees simultaneously making predictions. The baseline algorithm is summarized in Algorithm 1.

Algorithm 1 Baseline MultiTask Active Learning

```

1: procedure BASELINETRAIN( $D_{train}, D_{pool}, D_{test}, classifier, sampling$ )
2:   for each integer  $i = 1$  to  $n$  do
3:     if  $sampling$  then
4:        $D_{train} = SampleData(D_{train}, sampling)$ 
5:     end if
6:     Train  $classifier$  on  $D_{train}$  and make predictions on  $D_{pool}$ 
7:     Calculate performance metrics on  $D_{test}$ 
8:     Sort  $D_{pool}$  samples based on their level of confidence for all 3 targets
9:     Select  $m$   $D_{pool}$  samples each with the lowest level of confidence and add them to  $D_{train}$ 
10:  end for
11:  Plot  $accuracy$  and  $f1score$  as a function of query steps
12: end procedure

```

Independent Sampling

We use oversampling to mitigate the class imbalance in our datasets. Two of the sampling methods that we use extensively in our work are SMOTE [9] and Random Oversampling. Random Oversampling selects samples from the minority class and duplicates them in the training data. Synthetic Minority Oversampling Technique (SMOTE) on the other hand uses random data points from the minority class and their neighbors to generate synthetic samples for tackling data imbalance. This helps provide

new information that Random Oversampling fails to do. The term *Independent Sampling* has been used as we independently balance targets for the three tasks. A major drawback of this method is that while balancing one target, it can just so happen that we disrupt the balance for a different target. While training independent classifiers for each target, we avoid this problem as every classifier only fits its respective target and is blind to the imbalance in others. In a later section, when we deal with the simultaneous prediction of all targets, we propose a sampling method that takes this into account.

Algorithm 2 Independent Sampling

```

1: procedure SAMPLEDATA( $D_{train}, sampling$ )
2:   Let  $D_{train}$  be denoted as  $(X, y_1, y_2, y_3)$  where  $X$  is the data instance and  $y_1, y_2, y_3$  are respective targets
3:   for each integer  $i = 1$  to 3 do
4:      $D'_i = sampling(X, y_i)$ 
5:   end for
6:   Return  $(D'_1, D'_2, D'_3)$ 
7: end procedure

```

Adaptive Budget with Resampling

The next step was to use previous trends and allot the budget to target tasks more flexibly. This would help our framework focus more on tasks that are performing poorly. Relative f1 score difference for the last two query steps is considered while allotting the budget. Using relative differences makes it easier to compare performance boosts between tasks. Also, tasks that are performing better get penalized because of a larger denominator. We experiment with three methods to map percentage differences to the number of samples. These methods use functions like sigmoid and softmax for conversion into weights. The weights are multiplied by our budget to get the total number of samples. A separate approach uses fixed sample values (like 5, 15, 25), and we assign them according to the order of percentage differences for target tasks. So, for example, if we are working with three tasks (A, B, C) and the percentage F1 score differences are 10, 25, and 32, respectively. In this case, we will take 5 samples from task C, 15 from task B, and 25 from task A. This makes it so that more focus is put on the task that is at present performing the worst. A summary of the methods can be found in the algorithmic format below.

Algorithm 3 Adaptive Budget

```

1: procedure ADAPTIVETRAIN( $D_{train}, D_{pool}, D_{test}, classifier, sampling, adaptiveType$ )
2:   for each integer  $i = 1$  to  $n$  do
3:     if  $sampling$  then
4:        $D_{train} = SampleData(D_{train}, sampling)$ 
5:     end if
6:     if  $i = 1, 2$  then
7:       Follow steps 1 through 4 stated in Algorithm 1
8:     else
9:       Follow steps 1 through 2 stated in Algorithm 1
10:    end if
11:    Calculate the relative difference in F1 score values (denote it as  $x$ )
12:    if  $adaptiveType = softmax$  then
13:       $y = Softmax(-x)$ 
14:    else if  $adaptiveType = sigmoid$  then
15:       $y = Sigmoid(-x)$ 
16:    else if  $adaptiveType = Fixed$  then
17:      Fixed  $y$  values can be assigned after ordering  $x$ 
18:    end if
19:  end for
20:  Plot accuracy and f1 score as a function of query steps
21: end procedure

```

Multi-Output Implementation and Sampling

When working with independent classifiers, we ignore any available task overlap in the dataset. To exploit these relations, it is necessary to use an algorithm that can simultaneously predict all targets. To maintain consistency, we make use of the multi-output implementation of the Random Forest algorithm. This extension of the univariate random forest classifier uses

decision trees that consider average purity across all targets while posing splitting criteria. Having replaced the original classifiers with this one in all our previous implementations, we present our findings in the *Results* section. It should be noted that since we take the entirety of the data while fitting the classifier, it is challenging to balance all our targets separately. A novel approach is devised that considers appropriate regions to sample from and a proper sampling sequence. As a proof of concept, we implement our method on different datasets while restricting ourselves to only work with binary targets.

Consider there to be three targets as is in our work, each of which has only two classes, A and B. We first check for the individual local class imbalance by taking an absolute difference in the number of samples of one class from another. In a balanced target, this must be zero. We then order our targets in increasing magnitude of imbalance and start with the smallest one. Samples of the minority class for this target are randomly cloned to match that of the majority class. Having done that, we are now left with two targets to balance. The operation of cloning forces us to update values of imbalance previously calculated, and the remaining targets are reordered. The minority class samples of the following target are now split into two zones, one where the previously balanced target has class A, and the other has class B. An equal number of samples are cloned from both zones to ensure additional imbalance isn't injected. Now that two of the three targets are balanced, we can similarly balance the remaining target by considering four zones and sampling equally from each.

Results

Single-Output Training

Both baseline and adaptive budget methods are tested in our active learning framework. A total of 20 query steps are run, and we take note of the accuracy and f1 score trends for all of the datasets. The performance attained at the end of all query steps is compared across all baseline and adaptive budget methods to find the best one. Also, comparisons are made with and without the use of sampling. Following are the target-wise F1 score plots as a function of the query step for some of the datasets.

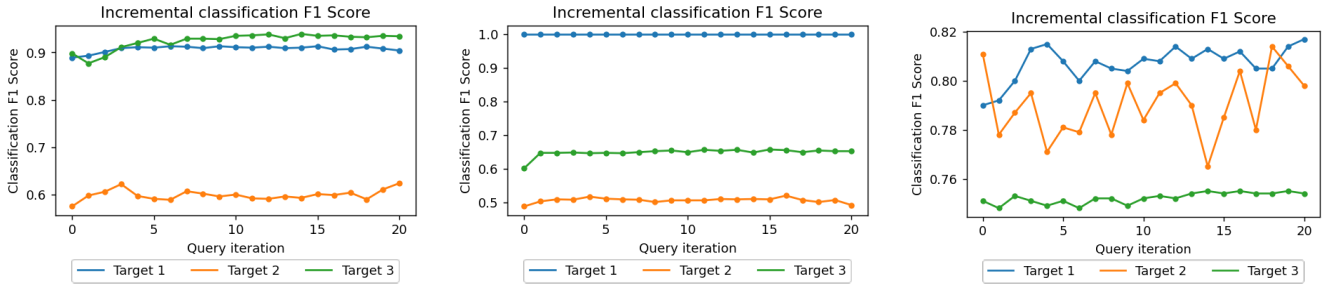


Figure 1. F1 score trends for the baseline implementation of IMDB, Mushroom, and Census dataset (from left to right)

It should be evident from the figures that a consistent trend is not seen. Some datasets show a steady increase for specific targets and, at the same time, decrease in others. Similar to the trend for the Census dataset, we often observe trends that oscillate in a narrow accuracy margin. We believe that this could be a cause of conflict of interest within targets for different samples. In other words, some samples carry information for a specific target which might allow them to be selected and sent to the oracle. Still, at the same time, these targets could also prove harmful to the performance of other targets.

We have tabulated and showcased below the dataset-wise final F1 scores across all single-output methods and sampling types. This helps check whether giving more liberty to the active learning framework in the form of an adaptive budget helped improve performance or not. Along with that, we are also able to understand better the effects of oversampling.

IMDB Single Output		F1 Score		
		No sampling	Random Oversampling	SMOTE
Target 1	Baseline	0.48	0.622	0.639
	Softmax	0.527	0.61	0.641
	Sigmoid	0.527	0.617	0.631
	Fixed	0.523	0.621	0.646
Target 2	Baseline	0.554	0.441	0.448
	Softmax	0.521	0.419	0.431
	Sigmoid	0.605	0.455	0.451
	Fixed	0.507	0.431	0.455
Target 3	Baseline	0.688	0.679	0.708
	Softmax	0.691	0.677	0.692
	Sigmoid	0.699	0.691	0.699
	Fixed	0.699	0.665	0.72

Mushroom Single Output		F1 Score		
		No sampling	Random Oversampling	SMOTE
Target 1	Baseline	1.0	1.0	1.0
	Softmax	1.0	1.0	1.0
	Sigmoid	1.0	1.0	1.0
	Fixed	1.0	1.0	1.0
Target 2	Baseline	0.484	0.551	0.553
	Softmax	0.48	0.556	0.55
	Sigmoid	0.479	0.55	0.554
	Fixed	0.4	0.554	0.549
Target 3	Baseline	0.581	0.745	0.737
	Softmax	0.541	0.745	0.736
	Sigmoid	0.542	0.745	0.743
	Fixed	0.544	0.745	0.745

Census Single Output		F1 Score		
		No sampling	Random Oversampling	SMOTE
Target 1	Baseline	0.711	0.789	0.769
	Softmax	0.718	0.766	0.768
	Sigmoid	0.731	0.782	0.778
	Fixed	0.733	0.753	0.771
Target 2	Baseline	0.358	0.437	0.466
	Softmax	0.37	0.421	0.39
	Sigmoid	0.394	0.418	0.414
	Fixed	0.359	0.394	0.44
Target 3	Baseline	0.164	0.225	0.218
	Softmax	0.161	0.207	0.214
	Sigmoid	0.158	0.228	0.223
	Fixed	0.158	0.217	0.225

References

1. Caruana, R. Multitask learning. *Mach. Learn.* **28**, 41–75, DOI: [10.1023/A:1007379606734](https://doi.org/10.1023/A:1007379606734) (1997).
2. Collobert, R. & Weston, J. A unified architecture for natural language processing: Deep neural networks with multitask learning. In *Proceedings of the 25th International Conference on Machine Learning, ICML '08*, 160–167, DOI: [10.1145/1390156.1390177](https://doi.org/10.1145/1390156.1390177) (Association for Computing Machinery, New York, NY, USA, 2008).
3. Majumder, N. *et al.* Sentiment and sarcasm classification with multitask learning. *IEEE Intell. Syst.* **34**, 38–43, DOI: [10.1109/MIS.2019.2904691](https://doi.org/10.1109/MIS.2019.2904691) (2019).
4. Luo, Y., Tao, D., Geng, B., Xu, C. & Maybank, S. J. Manifold regularized multitask learning for semi-supervised multilabel image classification. *IEEE Transactions on Image Process.* **22**, 523–536, DOI: [10.1109/TIP.2012.2218825](https://doi.org/10.1109/TIP.2012.2218825) (2013).
5. Yang, M. *et al.* Multitask learning for cross-domain image captioning. *IEEE Transactions on Multimed.* **21**, 1047–1061, DOI: [10.1109/TMM.2018.2869276](https://doi.org/10.1109/TMM.2018.2869276) (2019).
6. Wu, Z., Valentini-Botinhao, C., Watts, O. & King, S. Deep neural networks employing multi-task learning and stacked bottleneck features for speech synthesis. In *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 4460–4464, DOI: [10.1109/ICASSP.2015.7178814](https://doi.org/10.1109/ICASSP.2015.7178814) (2015).
7. Zhang, Z., Wu, B. & Schuller, B. Attention-augmented end-to-end multi-task learning for emotion prediction from speech. In *ICASSP 2019 - 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 6705–6709, DOI: [10.1109/ICASSP.2019.8682896](https://doi.org/10.1109/ICASSP.2019.8682896) (2019).
8. Zhang, Y. & Yang, Q. A survey on multi-task learning. *IEEE Transactions on Knowl. Data Eng.* 1–1, DOI: [10.1109/TKDE.2021.3070203](https://doi.org/10.1109/TKDE.2021.3070203) (2021).
9. Chawla, N. V., Bowyer, K. W., Hall, L. O. & Kegelmeyer, W. P. Smote: synthetic minority over-sampling technique. *J. artificial intelligence research* **16**, 321–357 (2002).
10. He, H., Bai, Y., Garcia, E. A. & Li, S. Adasyn: Adaptive synthetic sampling approach for imbalanced learning. In *2008 IEEE International Joint Conference on Neural Networks (IEEE World Congress on Computational Intelligence)*, 1322–1328, DOI: [10.1109/IJCNN.2008.4633969](https://doi.org/10.1109/IJCNN.2008.4633969) (2008).
11. Liu, X.-Y., Wu, J. & Zhou, Z.-H. Exploratory undersampling for class-imbalance learning. *IEEE Transactions on Syst. Man, Cybern. Part B (Cybernetics)* **39**, 539–550, DOI: [10.1109/TSMCB.2008.2007853](https://doi.org/10.1109/TSMCB.2008.2007853) (2009).
12. Sudre, C. H., Li, W., Vercauteren, T., Ourselin, S. & Jorge Cardoso, M. Generalised dice overlap as a deep learning loss function for highly unbalanced segmentations. In *Deep learning in medical image analysis and multimodal learning for clinical decision support*, 240–248 (Springer, 2017).
13. Charte, F., Rivera, A. J., del Jesus, M. J. & Herrera, F. Mlsmote: Approaching imbalanced multilabel learning through synthetic instance generation. *Knowledge-Based Syst.* **89**, 385–397 (2015).
14. Charte, F., Rivera, A. J., del Jesus, M. J. & Herrera, F. Dealing with difficult minority labels in imbalanced multilabel data sets. *Neurocomputing* **326**, 39–53 (2019).
15. Settles, B. From theories to queries: Active learning in practice. In *Active learning and experimental design workshop in conjunction with AISTATS 2010*, 1–18 (JMLR Workshop and Conference Proceedings, 2011).
16. Reker, D. Practical considerations for active machine learning in drug discovery. *Drug Discov. Today: Technol.* **32**, 73–79 (2019).
17. Dong, S. Multi class svm algorithm with active learning for network traffic classification. *Expert. Syst. with Appl.* **176**, 114885 (2021).
18. Naseem, U., Khushi, M., Khan, S. K., Shaukat, K. & Moni, M. A. A comparative analysis of active learning for biomedical text mining. *Appl. Syst. Innov.* **4**, 23 (2021).
19. Ertekin, S., Huang, J., Bottou, L. & Giles, L. Learning on the border: active learning in imbalanced data classification. In *Proceedings of the sixteenth ACM conference on Conference on information and knowledge management*, 127–136 (2007).
20. Torres, J. L. G., Catania, C. A. & Veas, E. Active learning approach to label network traffic datasets. *J. information security applications* **49**, 102388 (2019).
21. Zhang, Y. & Yang, Q. An overview of multi-task learning. *Natl. Sci. Rev.* **5**, 30–43 (2018).
22. Reichart, R., Tomanek, K., Hahn, U. & Rappoport, A. Multi-task active learning for linguistic annotations. In *Proceedings of ACL-08: HLT*, 861–869 (2008).

23. Zhang, Y. Multi-task active learning with output constraints. In *Twenty-Fourth AAAI Conference on Artificial Intelligence* (2010).
24. Xiao, Y., Chang, Z. & Liu, B. An efficient active learning method for multi-task learning. *Knowledge-Based Syst.* **190**, 105137 (2020).
25. Liu, B., Blekas, K. & Tsoumakas, G. Multi-label sampling based on local label imbalance. *Pattern Recognit.* **122**, 108294 (2022).